

Lab Assignment # 16

Course Title : **AI Assistant Coding**
Name of Student : **M.Anu Chandra**
Enrollment No. : **2303A54070**
Batch No. : **48**

Lab 16 – Database Design and Queries: Schema Design and SQL Generation

Task 1 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

- **Tables: Doctors, Patients, Appointments.**
- **Use AI to define constraints (unique IDs, valid dates).**
- **Generate queries:**
- **List all appointments for a specific doctor.**
- **Retrieve patient history by patient ID.**
- **Count total patients treated by each doctor.**

Expected Output:

- **Normalized schema and SQL queries with joins.**

```
studentdata.sql X
C: > Users > rgsga > Downloads > studentdata.sql > ...

1 CREATE TABLE Doctors (
2     doctor_id SERIAL PRIMARY KEY,
3     name VARCHAR(100) NOT NULL,
4     specialization VARCHAR(100),
5     phone VARCHAR(15) UNIQUE,
6     email VARCHAR(100) UNIQUE
7 );
8
9 CREATE TABLE Patients (
10    patient_id SERIAL PRIMARY KEY,
11    name VARCHAR(100) NOT NULL,
12    dob DATE NOT NULL,
13    gender VARCHAR(10),
14    phone VARCHAR(15) UNIQUE,
15    email VARCHAR(100) UNIQUE
16 );
17
18 CREATE TABLE Appointments (
19    appointment_id SERIAL PRIMARY KEY,
20    doctor_id INT NOT NULL,
21    patient_id INT NOT NULL,
22    appointment_date DATE NOT NULL,
23    diagnosis TEXT,
24    treatment TEXT,
25    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id) ON DELETE CASCADE,
26    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id) ON DELETE CASCADE,
27    CONSTRAINT valid_date CHECK (appointment_date >= '2000-01-01')
28 );
29
30 SELECT a.appointment_id, a.appointment_date, p.name AS patient_name, a.diagnosis, a.treatment
31 FROM Appointments a
32 JOIN Patients p ON a.patient_id = p.patient_id
33 WHERE a.doctor_id = 101; -- Replace 101 with the doctor's ID
34
35
36 SELECT a.appointment_id, a.appointment_date, d.name AS doctor_name, a.diagnosis, a.treatment
37 FROM Appointments a
```

doctor_id	name	specialization	phone	email
NULL	Dr. Arjun Rao	Cardiology	9876543210	arjun@hospital.com
NULL	Dr. Meera Joshi	Neurology	9876543211	meera@hospital.com
NULL	Dr. Sunil Patel	Orthopedics	9876543212	sunil@hospital.com

patient_id	name	dob	gender	phone	email
NULL	Ravi Kumar	1985-06-12	Male	9123456780	ravi@patient.com
NULL	Sita Devi	1990-02-20	Female	9123456781	sita@patient.com
NULL	Anil Sharma	1978-11-05	Male	9123456782	anil@patient.com

appointment_id	doctor_id	patient_id	appointment_date	diagnosis	treatment
NULL	1	1	2026-03-20	Flu	Rest + Medication
NULL	1	2	2026-03-22	Diabetes Check	Insulin Adjustment
NULL	2	3	2026-03-10	Migraine	Pain Relief Meds

appointment_id	doctor_id	patient_id	appointment_date	diagnosis	treatment
NULL	1	1	2026-03-20	Flu	Rest + Medication
NULL	1	2	2026-03-22	Diabetes Check	Insulin Adjustment
NULL	2	3	2026-03-10	Migraine	Pain Relief Meds

appointment_id	doctor_id	patient_id	appointment_date	diagnosis	treatment
NULL	1	1	2026-03-20	Flu	Rest + Medication
NULL	1	2	2026-03-22	Diabetes Check	Insulin Adjustment
NULL	2	3	2026-03-10	Migraine	Pain Relief Meds

appointment_id	doctor_id	patient_id	appointment_date	diagnosis	treatment
NULL	1	1	2026-03-20	Flu	Rest + Medication
NULL	1	2	2026-03-22	Diabetes Check	Insulin Adjustment
NULL	2	3	2026-03-10	Migraine	Pain Relief Meds

Task 2 – Real-Time Application: Online Shopping Database

Scenario: Design a database for an E-commerce platform.

Requirements:

- Tables: Users, Products, Orders, OrderDetails.
- Generate queries to:
 - Retrieve all orders by a user.
 - Find the most purchased product.
 - Calculate total revenue in a given month.
- AI should also suggest normalization improvements and query optimization.

Expected Output:

- Complete schema with relationships + SQL queries for analytics.

```
C:\Users> rgsga > Downloads > realtime.sql > ...
1 CREATE TABLE Users (
2   UserID INTEGER PRIMARY KEY AUTOINCREMENT,
3   Name TEXT NOT NULL,
4   Email TEXT UNIQUE NOT NULL,
5   PasswordHash TEXT NOT NULL,
6   CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
7 );
8
9
10 CREATE TABLE Products (
11   ProductID INTEGER PRIMARY KEY AUTOINCREMENT,
12   Name TEXT NOT NULL,
13   Description TEXT,
14   Price REAL NOT NULL,
15   Stock INTEGER NOT NULL,
16   CreatedAt TIMESTAMP DEFAULT CURRENT_TIMESTAMP
17 );
18
19 SELECT * FROM Users;
20 SELECT * FROM Products;
21 SELECT * FROM Orders;
22 SELECT * FROM OrderDetails;
23
24 CREATE TABLE Orders (
25   OrderID INTEGER PRIMARY KEY AUTOINCREMENT,
26   UserID INTEGER NOT NULL,
27   OrderDate TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
28   Status TEXT DEFAULT 'Pending',
29   FOREIGN KEY (UserID) REFERENCES Users(UserID)
30 );
31
32
33
34
35 CREATE TABLE OrderDetails (
36   OrderDetailID INTEGER PRIMARY KEY AUTOINCREMENT,
37   OrderID INTEGER NOT NULL,
```

SQL > 1 / 1 > 1 - 3 of 3

UserID	Name	Email	PasswordHash	CreatedAt
1	Alice Johnson	alice@example.com	hash123	2026-03-25 08:58:22
2	Bob Smith	bob@example.com	hash456	2026-03-25 08:58:22
3	Charlie Brown	charlie@example.com	hash789	2026-03-25 08:58:22

SQL > 1 / 1 > 1 - 4 of 4

ProductID	Name	Description	Price	Stock	CreatedAt
1	Laptop	15-inch, 16GB RAM, 512GB SSD	1200.0	10	2026-03-25 08:58:22
2	Smartphone	6.5-inch display, 128GB storage	800.0	25	2026-03-25 08:58:22
3	Headphones	Noise-cancelling wireless headphones	150.0	50	2026-03-25 08:58:22
4	Smartwatch	Fitness tracking smartwatch	200.0	30	2026-03-25 08:58:22

SQL > 1 / 1 > 1 - 4 of 4

OrderID	UserID	OrderDate	Status
1	1	2026-03-01 10:15:00	Completed
2	2	2026-03-05 14:30:00	Completed
3	1	2026-03-10 09:45:00	Pending
4	3	2026-03-15 16:20:00	Completed

SQL > 1 / 1 > 1 - 6 of 6

OrderDetailID	OrderID	ProductID	Quantity	Price
1	1	1	1	1200.0

```
studentdata.sql x realtime.sql x
C:\Users> rgsga > Downloads > realtime.sql > ...
51 JOIN OrderDetails od ON o.OrderID = od.OrderID
52 JOIN Products p ON od.ProductID = p.ProductID
53 WHERE o.UserID = 101; -- Example UserID
54
55
56
57 SELECT p.ProductID, p.Name, SUM(od.Quantity) AS TotalPurchased
58 FROM OrderDetails od
59 JOIN Products p ON od.ProductID = p.ProductID
60 GROUP BY p.ProductID, p.Name
61 ORDER BY TotalPurchased DESC
62 LIMIT 1;
63
64
65 SELECT strftime('%Y-%m', o.OrderDate) AS Month,
66        SUM(od.Quantity * od.Price) AS TotalRevenue
67 FROM Orders o
68 JOIN OrderDetails od ON o.OrderID = od.OrderID
69 WHERE strftime('%Y-%m', o.OrderDate) = '2026-03'
70 GROUP BY Month;
71
72
73
74
75 INSERT INTO Users (Name, Email, PasswordHash)
76 VALUES
77 ('Alice Johnson', 'alice@example.com', 'hash123'),
78 ('Bob Smith', 'bob@example.com', 'hash456'),
79 ('Charlie Brown', 'charlie@example.com', 'hash789');
80
81
82 INSERT INTO Products (Name, Description, Price, Stock)
83 VALUES
84 ('Laptop', '15-inch, 16GB RAM, 512GB SSD', 1200.00, 10),
85 ('Smartphone', '6.5-inch display, 128GB storage', 800.00, 25),
86 ('Headphones', 'Noise-cancelling wireless headphones', 150.00, 50),
87 ('Smartwatch', 'Fitness tracking smartwatch', 200.00, 30);
```

SQLite x

2	Bob Smith	bob@example.com	hash456	2026-03-25 08:58:22
3	Charlie Brown	charlie@example.com	hash789	2026-03-25 08:58:22

SQL > 1 / 1 > 1 - 4 of 4

ProductID	Name	Description	Price	Stock	CreatedAt
1	Laptop	15-inch, 16GB RAM, 512GB SSD	1200.0	10	2026-03-25 08:58:22
2	Smartphone	6.5-inch display, 128GB storage	800.0	25	2026-03-25 08:58:22
3	Headphones	Noise-cancelling wireless headphones	150.0	50	2026-03-25 08:58:22
4	Smartwatch	Fitness tracking smartwatch	200.0	30	2026-03-25 08:58:22

SQL > 1 / 1 > 1 - 4 of 4

OrderID	UserID	OrderDate	Status
1	1	2026-03-01 10:15:00	Completed
2	2	2026-03-05 14:30:00	Completed
3	1	2026-03-10 09:45:00	Pending
4	3	2026-03-15 16:20:00	Completed

SQL > 1 / 1 > 1 - 6 of 6

OrderDetailID	OrderID	ProductID	Quantity	Price
1	1	1	1	1200.0
2	1	3	2	150.0
3	2	2	1	800.0
4	2	4	1	200.0
5	3	3	1	150.0
6	4	2	2	800.0

Task 3 – Library Database

Task: Design schema for a Library Management System.

Instructions:

- **Tables:** Books, Members, Loans.
 - Use AI to suggest indexing strategy for faster queries.
 - **Generate queries:**
 - Retrieve all books currently issued.
 - Find overdue books (loan date > 30 days).
 - Count number of books loaned by each member.
- Expected Output:**
- Schema + SQL queries demonstrating joins and conditions.

studentdata.sql × realtime.sql × Library.sql ×

C:\Users> rgsa > Downloads > Library.sql > SELECT * FROM Books;

1 CREATE TABLE Books (
2 BookID INT PRIMARY KEY,
3 Title VARCHAR(255) NOT NULL,
4 Author VARCHAR(255),
5 ISBN VARCHAR(20) UNIQUE,
6 PublishedYear INT,
7 Available BOOLEAN DEFAULT TRUE
8);
9
10 CREATE TABLE Members (
11 MemberID INT PRIMARY KEY,
12 Name VARCHAR(255) NOT NULL,
13 Email VARCHAR(255) UNIQUE,
14 Phone VARCHAR(15),
15 MembershipDate DATE
16);
17
18
19 CREATE TABLE Loans (
20 LoanID INT PRIMARY KEY,
21 BookID INT REFERENCES Books(BookID),
22 MemberID INT REFERENCES Members(MemberID),
23 LoanDate DATE NOT NULL,
24 ReturnDate DATE,
25 DueDate DATE NOT NULL
26);
27
28 CREATE INDEX idx_books_available ON Books(Available);
29 CREATE INDEX idx_loans_book_member ON Loans(BookID, MemberID);
30 CREATE INDEX idx_loans_due ON Loans(DueDate);
31
32 SELECT b.BookID, b.Title, m.Name AS Borrower, l.LoanDate
33 FROM Books b
34 JOIN Loans l ON b.BookID = l.BookID
35 JOIN Members m ON l.MemberID = m.MemberID
36 WHERE b.Available = FALSE AND l.ReturnDate IS NULL;
37

SQLite ×

SQL v < 1 / 1 > 1 - 5 of 5

BookID	Title	Author	ISBN	PublishedYear	Available
1	The Great Gatsby	F. Scott Fitzgerald	9780743273565	1925	1
2	1984	George Orwell	9780451524935	1949	1
3	To Kill a Mockingbird	Harper Lee	9780061120084	1960	1
4	The Catcher in the Rye	J.D. Salinger	9780316769488	1951	1
5	Pride and Prejudice	Jane Austen	9780141439518	1813	1

SQL v < 1 / 1 > 1 - 4 of 4

LoanID	BookID	MemberID	LoanDate	ReturnDate	DueDate
1001	1	101	2026-02-20	NULL	2026-03-21
1002	2	102	2026-01-15	2026-02-10	2026-02-14
1003	3	103	2026-02-01	NULL	2026-03-02
1004	4	104	2026-03-10	NULL	2026-04-09

SQL v < 1 / 1 > 1 - 4 of 4

MemberID	Name	Email	Phone	MembershipDate
101	Alice Johnson	alice@example.com	9876543210	2023-01-15
102	Bob Smith	bob@example.com	9876543211	2023-02-20
103	Charlie Brown	charlie@example.com	9876543212	2023-03-05
104	Diana Prince	diana@example.com	9876543213	2023-04-10

Task 4: Optimize Queries

Instructions:

- Use AI tools to analyze query efficiency.
- Optimize inefficient queries using joins, indexes, or reducing subqueries.
- Test optimized queries for correctness.

Starter Code Example:

-- Original query

```
SELECT * FROM Books WHERE author_id IN (SELECT author_id FROM Authors WHERE country='UK');
```

-- Optimized query

```
SELECT b.*  
FROM Books b  
JOIN Authors a ON b.author_id = a.author_id  
WHERE a.country = 'UK';
```

Expected Output:

- Both queries return the same result: all books written by UK authors.
- Optimized query performs faster for larger datasets.

SQL v < 1 / 1 > 1 - 3 of 3			
BookID	Title	Borrower	LoanDate
1	The Great Gatsby	Alice Johnson	2026-02-20
3	To Kill a Mockingbird	Charlie Brown	2026-02-01
4	The Catcher in the Rye	Diana Prince	2026-03-10

